Machine Learning Project Guidelines

Each team builds, evaluates and compares machine learning models on three types of data (numerical, image and text) and wraps them in a simple GUI.

1. Overview

You will work as a team to solve three machine learning problems end to end: a regression problem on numerical data, a classification problem on images, and a classification problem on text. For each problem you prepare the data, train two models, tune them, evaluate them on unseen data, and explain which model works better and why. Finally, you connect your trained models to a simple GUI.

By the end of the project you should be able to:

1. Explore and clean a real dataset (missing values, scaling, encoding).
2. Turn images and text into numerical features a model can use.
3. Split data correctly into training, validation and test sets.
4. Tune hyperparameters using the validation set only.
5. Choose the right evaluation metrics for regression and for classification.
6. Compare models and justify your conclusions with evidence.
7. Save a trained model and use it to make predictions on new inputs.

2. Teams and timeline

Item	Details
Team size	6 to 7 students
Team leader	One per team; submits the forms and the repository link
Final submission	GitHub repository + cover sheet
Discussion	every team member must attend

3. Project tasks

Part A: Numerical data (Regression)

Predict a continuous numerical value (for example a price or a cost).

- **Models:** Linear Regression and KNN Regressor.
- **Required steps:**

  - Short exploratory data analysis (EDA): summary statistics, histograms, correlations.
  - Preprocessing: handle missing values, encode categorical columns, scale features.
  - Tune at least one hyperparameter using the validation set (for example k in KNN).
- **Metrics on the test set:** MAE, RMSE and R^2, plus a Predicted vs Actual plot for each model.

Part B: Image data (Classification)

Classify images into **3 to 5 classes**.

- **Models:** Logistic Regression and KNN Classifier.
- **K-Means (clustering):** apply K-Means then map each cluster to the most frequent true label inside it to compute accuracy. Also report the Silhouette score. K-Means is an unsupervised algorithm, so explain how it differs from the two classifiers.
- **Required steps:**
  - Resize all images to one fixed size, then normalize pixel values.
  - Feature extraction: raw pixels (flattened), or better features such as HOG, color histograms, or PCA on the pixels.
  - Tune at least one hyperparameter per model using the validation set.
- **Metrics on the test set:** Accuracy, Confusion Matrix, and Precision, Recall and F1-score per class.

Part C: Text data (Classification)

Classify texts into categories (for example spam vs not spam, or news topics).

- **Models:** Logistic Regression and Linear SVM.
- **Required steps:**
  - Text cleaning: lowercasing, removing punctuation, numbers and stopwords, and any HTML tags.
  - Feature extraction: Bag of Words or TF-IDF (state the vocabulary size you used).
  - Tune at least one hyperparameter per model using the validation set.
- **Metrics on the test set:** Accuracy, Confusion Matrix, and Precision, Recall and F1-score per class.

Part D: GUI (mandatory)

Build a simple GUI (for example Streamlit, Gradio or Tkinter) where the user enters numerical values, uploads an image, or types a text, and gets your model's prediction. Load your saved models; do not retrain inside the GUI.

Bonus

Extra marks are given for any of the following:

- **Ensemble approach:** combine models with Voting, Bagging Boosting or Stacking, in any part. To count, you must compare the ensemble with your base models on the same test set and explain in the discussion how it works and why it did or did not improve the results.
- **Implementation from scratch:** implement one of the models with NumPy only (for example Linear Regression or Logistic Regression with gradient descent) and compare it with the scikit-learn version.
- **Arabic text:** use an Arabic text dataset for Part C (with language preprocessing).

4. Technical rules

- **Data splitting:** split every dataset into training, validation and test sets (for example 70% / 15% / 15%). Cross-validation on the training data is also accepted instead of a fixed validation set. For classification, use a stratified split so each class keeps its proportion.
- **Use of the test set:** choose all hyperparameters using the validation set (or cross-validation). Evaluate on the test set **only once**, at the very end. Never fit a scaler, vectorizer or PCA on the test data; fit them on the training data only.
- **Reproducibility:** fix `random_state` in every split and model, so the TA gets the same results when running your code.

5. Datasets

Each team picks one dataset per part.

Requirements:

- **Use the raw data, not a cleaned version.** Preprocessing (handling missing values, cleaning, encoding, scaling, resizing, text cleaning) is an important part of the project and is graded. Do not use a dataset that was already cleaned or preprocessed by someone else. Use the original dataset as released by its source. Pre-organized class folders are acceptable, but students must perform the required preprocessing themselves.
- **Not allowed:** datasets used in the sections (California Housing, MNIST, scikit-learn's `load_digits`).
- **Numerical:** at least 1000 rows; the target must be a continuous number, not categories.
- **Images:** 3 to 5 classes, at least 100 images per class, classes roughly balanced. If a dataset has more than 5 classes, choose 5 of them and say which.
- **Text:** at least 1,000 texts with labels.

Suggested datasets (you may choose others that meet the requirements):

Part	Dataset	Notes
Numerical	Medical Insurance Cost	Predict insurance charges
Numerical	Bike Sharing (UCI)	Predict the number of rentals
Numerical	Student Performance (UCI)	Predict the final grade
Numerical	Car price datasets (Kaggle)	Predict the selling price
Images	Flowers Recognition (Kaggle)	5 flower classes
Images	Intel Image Classification (Kaggle)	6 scene classes: choose 5
Images	Fruits-360	Choose 3 to 5 fruits
Text	SMS Spam Collection (UCI)	2 classes: spam / ham
Text	BBC News	5 topic classes
Text	20 Newsgroups (built into scikit-learn)	Choose up to 5 categories
Text	IMDB Reviews	2 classes; a subset is enough

6. Deliverables

6.1 GitHub repository

- One repository per team. The team leader submits the repository link through the submission form (before the discussion).
- **Every team member must commit their own work** throughout the project. The commit history is checked during the individual assessment; a repository uploaded in one commit the night before the discussion loses marks.
- Suggested structure: `part_a_numerical/`, `part_b_images/`, `part_c_text/`, `gui/`, `models/` (saved models), `README.md`, `requirements.txt`.
- **Datasets:** upload the dataset if it is small. for large datasets put a download link (Kaggle or Google Drive) in the README.
- The README explains how to install the requirements, run each notebook, and launch the GUI.

6.2 Project cover sheet and submission form

Both will be released before the discussion:

- **Cover sheet:** faculty name, course name, team number, and every member's name and student ID.
- **Submission form:** filled in by the team leader, including the link to the team's GitHub repository.

6.3 Project report

One report covering the three parts. For **each part (A, B and C)** include:

1. **Dataset information:** name and source link; number of samples; for classification, the number of classes, their labels and the number of samples per class; for images, the image size (original and after resizing); the number of samples in training, validation and testing.
2. **Preprocessing and feature extraction:** the steps you applied; how many features you extracted, their names or type (for example HOG, TF-IDF), and the final feature dimension.
3. **Models and hyperparameters:** for each model, the hyperparameters you tried, how you chose them on the validation set, and the final values.
4. **Results on the test set:**
   - Part A: MAE, RMSE and R^2, and a Predicted vs Actual plot for each model.
   - Parts B and C: Accuracy, Confusion Matrix, and Precision, Recall and F1-score per class for each model. For K-Means, also the cluster-to-label mapping and the Silhouette score.
5. **Comparison and discussion:** which model performed better and why, where the models make mistakes (for example which classes they confuse).

Then add:

- **GUI:** screenshots of each tab making a prediction.
- **Contribution table:** what each team member did.

7. Discussion and individual assessment

Every team member is assessed individually during the discussion.

- **Any member can be asked about any part of the project**, not only the part they worked on. Each member should understand the whole project.
- You may be asked to explain any line of your code, and to **make a small change live** (for example change k, add a metric, or change the image size) and explain the effect.
- Typical questions: why you chose a preprocessing step, how a model works, why one model beat the other, what a confusion matrix shows.
- Part of your grade is individual and reflects how well you explain the work.
- **A student who cannot explain any part of the project may receive a reduced share of the team's marks** (for example 50% or other), at the TA's discretion, regardless of the team's result.
- **If no team member can explain the submitted work, the whole project receives 0.**
- Missing the discussion without an accepted excuse means 0 for the individual assessment.

8. Academic integrity

The goal of this project is that **you** learn to build machine learning models. Submitting work you did not write defeats that goal.

Not allowed:

- **Copying code** from another team, from previous years' projects, or from online notebooks (Kaggle, GitHub, blogs) and submitting it as your own.
- **Using AI tools to write your code**, such as ChatGPT, Claude, Gemini, GitHub Copilot or similar.
- Sharing your code with another team. Both the team that copies and the team that shares are penalized.

Allowed:

- Official documentation (scikit-learn, pandas, OpenCV, NLTK) and the course lectures and section notebooks.
- Asking the TA for help.
- Asking an AI tool to explain a concept or an error message, as long as it does not write your code.

Consequences:

- Code that is copied, or that you cannot explain line by line during the discussion, is treated as not your own work.
- This can lead to 0 for the affected part, or 0 for the whole project in serious cases.
- Code similarity between teams will be checked.

GENERAL NOTES:

- The preprocessing steps listed in each part are a guide, not a checklist. Not every step is required for every dataset: apply the steps your data needs. However, a submission with no preprocessing at all is not accepted.
- Your accuracy or error values are not graded on their own. Marks come from correct methodology and clear, justified analysis.
- The most important thing is to justify every step: explain why you applied it and what problem it solves in your data. Also explain why you skipped a step that might be expected.